QADQN: Quantum Attention Deep Q-Network for Financial Market Prediction

Siddhant Dutta[1], Nouhaila Innan[2,3], Alberto Marchisio[2,3], Sadok Ben Yahia[4], and Muhammad Shafique[2,3]
[1]SVKM's Dwarakdas J. Sanghvi College of Engineering, India
[2]eBRAIN Lab, Division of Engineering, New York University Abu Dhabi (NYUAD), Abu Dhabi, UAE
[3]Center for Quantum and Topological Systems (CQTS), NYUAD Research Institute, NYUAD, Abu Dhabi, UAE
[4]The Maersk Mc-Kinney Moller Institute, University of Southern Denmark, Alsion 2, 6400- Sønderborg, Denmark
siddhant.dutta180@svkmmumbai.onmicrosoft.com, nouhaila.innan@nyu.edu, alberto.marchisio@nyu.edu, say@mmmi.sdu.dk
muhammad.shafique@nyu.edu,

arXiv:2408.03088v1 [quant-ph] 6 Aug 2024

***Abstract*—Financial market prediction and optimal trading strategy development remain challenging due to market complexity and volatility. Our research in quantum finance and reinforcement learning for decision-making demonstrates the approach of quantum-classical hybrid algorithms to tackling real-world financial challenges. In this respect, we corroborate the concept with rigorous backtesting and validate the framework's performance under realistic market conditions, by including fixed transaction cost per trade. This paper introduces a Quantum Attention Deep Q-Network (QADQN) approach to address these challenges through quantum-enhanced reinforcement learning. Our QADQN architecture uses a variational quantum circuit inside a traditional deep Q-learning framework to take advantage of possible quantum advantages in decision-making. We gauge the QADQN agent's performance on historical data from major market indices, including the S&P 500. We evaluate the agent's learning process by examining its reward accumulation and the effectiveness of its experience replay mechanism. Our empirical results demonstrate the QADQN's superior performance, achieving better risk-adjusted returns with Sortino ratios of 1.28 and 1.19 for non-overlapping and overlapping test periods respectively, indicating effective downside risk management.**

***Index Terms*—Quantum Reinforcement Learning, Quantum Finance, Decision-Making**

I. Introduction

Financial markets are characterized by their complexity and unpredictability, driven by many factors that fuel volatility and challenge the efficacy of predictive models. Traditional methods in computational finance often struggle to fully utilize the vast datasets available due to limitations in computational power and algorithm complexity [1]. The complexity arises from various interacting elements such as macroeconomic indicators, market sentiment, and geopolitical events contributing to market volatility [2]–[4].

Volatility in financial markets poses significant challenges for both prediction and risk management. Traditional volatility models, like GARCH and its variants, have been widely used but often fall short of capturing market behaviors' intricate and dynamic nature. The limitations of these models highlight the need for more advanced approaches that can handle non-linearities and high-dimensional data.

Financial statement data, alongside information extracted from business news, can be integrated with advanced ML algorithms to generate investment signals or predict a company's future performance [5]. This approach enhances the stock screening process by identifying promising investment opportunities. However, while these algorithms effectively address stock selection, they do not inherently resolve the issue of optimal position sizing and allocation among the chosen investments. Consequently, the trader must still exercise discretion in determining the precise timing for entering and exiting positions [6].

Current deep learning methodologies, such as Recurrent Neural Networks (RNNs), Long Short-Term Memory Networks (LSTMs), and Transformer models, have been employed to predict stock prices by learning from historical data. These models excel at capturing temporal dependencies and patterns in the data, leading to improved predictive performance. However, they often require extensive computational resources and can be prone to overfitting, especially when dealing with noisy and non-stationary financial data [7], [8].

In addition to employing Machine Learning (ML) and deep learning techniques, traditional stochastic process models such as Geometric Brownian Motion (GBM) offer a reliable method for stock price prediction by simulating market returns as a continuous-time random walk. The GBM model posits that the logarithm of stock prices follows a normal distribution, thereby incorporating the inherent randomness and volatility of financial markets. However, this assumption also presents a limitation, as it may not fully capture extreme market events or non-linear dynamics [9], [10]. The significance of GBM extends to its foundational role in the development of the Black-Scholes option pricing model, which is extensively applied in the valuation of financial derivatives [11]. This model's simplicity and interpretability make it a valuable tool for stock price prediction, provided historical price data is available [12].

The QADQN framework addresses the limitations of computational overhead in attention mechanisms by replacing them with a Multi-Head Quantum Attention layer. It also improves explainability by integrating quantum attention mechanisms, utilizing the computational power of quantum computing to enhance the processing and interpretation of large-scale financial data. Using imitative learning strategies helps to solve the problem of finding a balance between exploitation and exploration [13]. The trading agent is instructed via the use of a Q-learning algorithm and a replay buffer that is initially filled with actions taken from the Dual Thrust approach [14]. These methods provide the network with enhanced expertise

in the financial area when they are integrated into the POMDP framework.

The novel contributions of this work can be summarized as follows:

- Incorporation of *quantum attention layers* within deep Reinforcement Learning (RL) agents to enhance the framework's ability to focus on relevant market features, resulting in improved decision-making efficiency.
- Application of the QADQN within a *Partially Observable Markov Decision Process (POMDP) framework* utilizing attention-based deterministic policy gradient method, which is well-suited for dealing with the uncertainty and partial observability inherent in financial markets.
- Through modified Q-learning and the initialization of a replay buffer, pre-populated with actions derived from the established Dual Thrust methodology, imitative learning approaches in quantum agents aim to maintain an optimal balance between exploration and exploitation of available resources.
- Backtesting and validation against historical market data, including major indices like the S&P 500. The backtesting process includes considerations for transaction costs and estimates the total return rate, Maximum Drawdown, and the final Sharpe and Sortino ratio, providing an assessment of the framework's reliability.
- Development of an attention deterministic policy gradient method tailored to handle the POMDP setting for quantum agents. This method enhances the framework's ability to learn effective trading strategies and trends, thereby improving its long-term reward accumulation.

Sec. II reviews Quantum Computing (QC) and RL in financial markets; Sec. III explains the QADQN, POMDP, and dual thrust strategy; Sec. IV compares QADQN's performance with prior methods and different market scenarios; Sec. V concludes with key findings and future directions for quantum-enhanced RL in trading.

II. Background and Related Work

A. Quantum Computing for Finance

Recent advancements in QC have demonstrated significant potential for financial market applications, particularly in enhancing prediction accuracy and decision-making efficiency. Quantum Machine Learning (QML) emerges as a promising field [15], [16], combining the computational power of quantum systems with ML algorithms to analyze financial data more effectively [17], [18].

Consider a quantum state representing a financial portfolio:

$$|\psi_{\text{portfolio}}\rangle = \sum_i \alpha_i |a_i\rangle, \tag{1}$$

where $|a_i\rangle$ represents individual assets and α_i their corresponding weights [19], subject to the normalization condition $\sum_i |\alpha_i|^2 = 1$.

Portfolio optimization can be formulated as a Quadratic Unconstrained Binary Optimization (QUBO) problem:

$$\min f(x) = x^T Q x + c^T x, \tag{2}$$

where x is a binary vector representing asset selection, Q is the covariance matrix, and c represents expected returns. This can be mapped to a quantum Hamiltonian:

$$H = \sum_{ij} Q_{ij} \sigma_i^z \sigma_j^z - \sum_i c_i \sigma_i^z, \tag{3}$$

where σ_i^z are Pauli-Z operators. Quantum algorithms like Quantum Approximate Optimization Algorithm (QAOA) can be applied to find the ground state of this Hamiltonian, representing the optimal portfolio allocation:

$$|\psi_{\text{QAOA}}\rangle = \prod_p e^{-i\beta_p H_B} e^{-i\gamma_p H_C} |\psi_{\text{init}}\rangle, \tag{4}$$

where H_B and H_C are mixing and cost Hamiltonians respectively, and β_p, γ_p are variational parameters [20].

For risk assessment, quantum entanglement can model complex correlations. Value-at-Risk (VaR) is a widely used measure in risk management to quantify the potential loss on an asset or portfolio over a specific time horizon and with a certain confidence level:

$$\text{VaR}_\alpha[L] = \inf\{p \in \mathbb{R}^+ : \langle\psi|\hat{P}_L \leq p|\psi\rangle \geq \alpha\}, \tag{5}$$

where $\hat{P}_L \leq p$ is a projection operator onto the subspace where the loss is less than or equal to p.

Conditional Value-at-Risk (CVaR) [21], or expected shortfall, is the expected loss given that the loss exceeds the VaR threshold. It can be expressed as:

$$|\psi_{\text{VaR}}\rangle = \frac{\hat{P}_{L>\text{VaR}_\alpha}|\psi\rangle}{\sqrt{\langle\psi|\hat{P}_{L>\text{VaR}_\alpha}|\psi\rangle}}, \tag{6}$$

$$\text{CVaR}_\alpha[L] = \langle\psi_{\text{VaR}}|\hat{L}|\psi_{\text{VaR}}\rangle. \tag{7}$$

The Economic Capital Requirement (ECR) is defined as the difference between VaR and the expected loss:

$$\text{ECR}_\alpha[L] = \text{VaR}_\alpha[L] - \langle\psi|\hat{L}|\psi\rangle. \tag{8}$$

B. Quantum Machine Learning for Finance

QML algorithms, such as quantum neural networks, can be used for financial time series prediction [22], [23]. These quantum algorithms leverage superposition and entanglement to process large financial datasets more efficiently. One promising field within QML is Quantum Reinforcement Learning (QRL) [24], which integrates QC with RL to develop advanced trading strategies. In QRL, quantum circuits are used to enhance exploration and exploitation mechanisms within RL frameworks. The basic RL model can be represented as:

$$Q^*(s,a) = \max_\pi \mathbb{E}_\pi \left[\sum_{t=0}^{\infty} \gamma^t r_{t+1} \mid S_t = s, a_t = a \right], \tag{9}$$

where $Q^*(s,a)$ is the optimal action-value function, γ is the discount factor, r_t is the reward at time t, and π is the policy.

However, integrating quantum circuits into RL frameworks poses significant challenges. For instance, Quantum Deep Q-Networks (QDQN) and Quantum Policy Gradient methods have shown theoretical benefits, but issues like quantum noise,

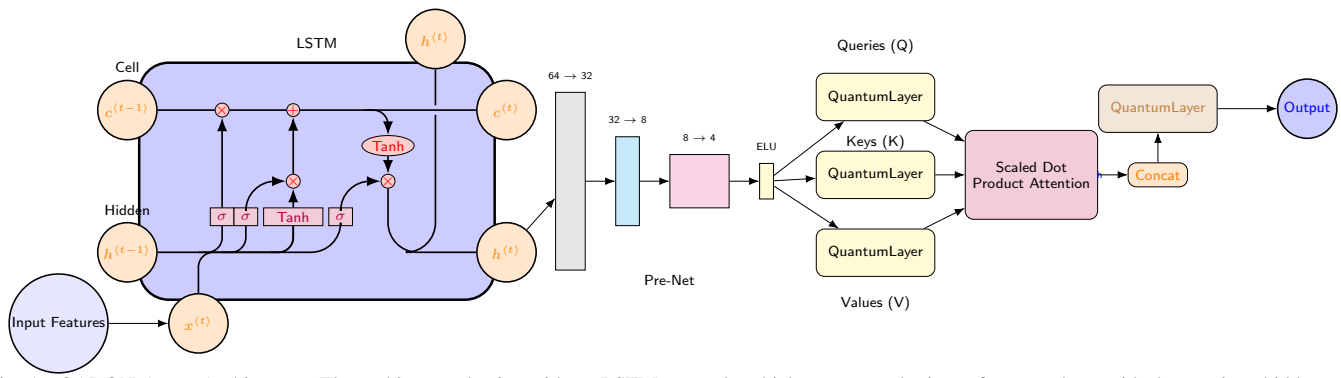

Fig. 1. QADQN Agent Architecture: The architecture begins with an LSTM network, which processes the input features along with the previous hidden and cell states $(h(t-1), c(t-1))$ to produce the current hidden and cell states $(h(t), c(t))$. The output of the LSTM is then passed through a pre-net consisting of linear layers reducing the dimensionality from 64 to 32, then to 8, and finally to 4, which is the number of qubits. These outputs are fed into quantum layers to generate the Queries (Q), Keys (K), and Values (V) for the scaled dot-product attention mechanism. The attention mechanism integrates these quantum-derived components, followed by concatenation and an additional quantum layer to output Q-values, which are used for action selection in OHLC data processing.

decoherence, and scalability impede practical implementation [25]. Furthermore, the optimization of VQC is represented as:

$$\theta^* = \arg\min_{\theta} \langle \psi(\theta) \mid H \mid \psi(\theta) \rangle, \tag{10}$$

and it requires careful tuning of quantum parameters θ, which can be computationally intensive in terms of convergence. Recent advancements in quantum attention mechanisms have demonstrated potential in enhancing the performance of ML, including computer vision and high energy physics tasks [26]–[28]. These mechanisms leverage QC principles to efficiently handle the high-dimensional parameter spaces associated with self-attention, a crucial component in many contemporary deep-learning architectures [29].

III. QADQN Framework

The Quantum Attention Deep Q-Network (QADQN) framework extends the concept of hybrid quantum-classical architectures to the domain of RL. By integrating NISQ-based quantum self-attention mechanisms with deep Q-networks [30]–[33], QADQN aims to improve decision-making processes in complex environments. Our approach involves carefully optimizing Variational Quantum Circuit (VQC) parameters using the backpropagation method for gradient computation, addressing convergence challenges in high-dimensional quantum-classical hybrid systems in the financial domain.

The input to the QADQN is an n-day state representation of financial data transformed using a logarithmic function, denoted as $S_t \in \mathbb{R}^{n \times f}$, where n is the number of days, and f is the number of features per day. Each row of the state S_t at time t is given by:

$$S_{t,i} = \left[\ln(p_{t-i+1}/p_{t-i}), \ldots, \ln(f^f_{t-i+1}/f^f_{t-i}) \right] \tag{11}$$

where p_t represents the close price at time t, f^j_t represents the j-th open-high-low-close (OHLC)-based feature at time t, and i ranges from 1 to n, covering the n-day window.

A. Pre-Neural Net

The LSTM layer processes the input sequence and outputs a sequence of hidden states $h_t \in \mathbb{R}^{64}$ for each time step. For single-step predictions, we use the last hidden state, while for sequence outputs, we use the full sequence of hidden states [34], as shown in Fig. 1:

$$h_t, c_t = \text{LSTM}(S_t, h_{t-1}, c_{t-1}), \tag{12}$$

where h_t is the hidden state and c_t is the cell state at time t.

$$x_t = \text{ELU}(W_L \cdot \text{ELU}(W_{L-1} \cdot \ldots \text{ELU}(W_1 \cdot h_t + b_1) + b_{L-1}) + b_L). \tag{13}$$

While attention mechanisms, similar to those in transformer models, can be employed independently, our hybrid approach enables the capture of long-term dependencies by extracting temporal features with LSTM and refining the LSTM's contextualized output with the quantum attention mechanism.

B. Quantum Attention Mechanism

The core of our framework is the Quantum Multihead Self-Attention (QMSA) mechanism. This mechanism projects the input state to a quantum state using parameterized quantum circuits, computes attention scores, and aggregates the results:

a) Data Encoding

Each input row x_i of the LSTM output is encoded into a quantum state using a data loader operator $U^\dagger(x_i)$:

$$|x_i\rangle \equiv U^\dagger(x_i)|0\rangle = \bigotimes_{j=1}^{dh} R^\dagger_x(x_{ij})|0\rangle, \tag{14}$$

where R_x is a parameterized rotation around the x-axis and dh represents the dimension of the hidden state.

b) Key and Query Operations

For each input row x_i, key operator $K^\dagger(\theta_K)$ and query operator $Q^\dagger(\theta_Q)$ are applied:

$$K_i = \langle x_i | K^\dagger(\theta_K) Z_0 K(\theta_K) | x_i \rangle, \tag{15}$$

$$Q_i = \langle x_i | Q^\dagger(\theta_Q) Z_0 Q(\theta_Q) | x_i \rangle, \tag{16}$$

where Z_0 represents a spin measurement of the qubit in the z-direction.

c) Value Operation

Each row of the input is passed through a value operator $V^\dagger(\theta_V)$ to obtain the value matrix V:

$$V_{ij} = \langle x_i | V^\dagger(\theta_V) Z_j V(\theta_V) | x_i \rangle. \tag{17}$$

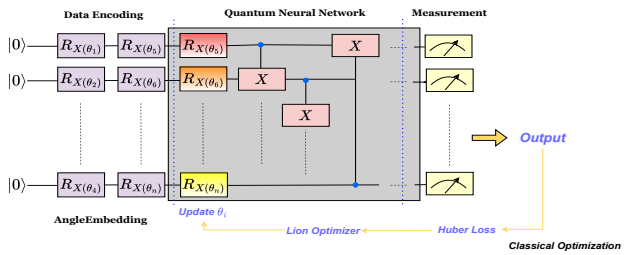

Fig. 2. Quantum layer circuit: It begins with data encoding using angle embedding, where input data is encoded into the quantum state via parameterized rotation gates $R_X(\theta_i)$. The encoded quantum states are then processed by a parameterized quantum circuit consisting of additional parameterized rotation gates and Controlled-NOT gates, facilitating entanglement and complex quantum state manipulation. The quantum states are subsequently measured to obtain classical outputs. During training, the classical outputs compute the Huber loss, and the parameters θ_i are updated using the Lion optimizer.

The circuit used to create K, Q, and V is shown in Figure 2.

d) Hybrid Attention Head Computation

The attention matrix A is computed using key and query vectors as $A_{ij} = -(Q_i - K_j)^2$. The final output is determined by applying softmax to A normalized by $\sqrt{dh}$, then multiplying the result by the value matrix V, yielding $\text{SoftMax}\left(\frac{A}{\sqrt{dh}}\right) \cdot V$.

C. Quantum Post-Net Layer

The VQC is defined using the PennyLane framework. The circuit consists of an angle embedding layer followed by entangling layers as shown in Fig. 2:

$$|\psi(\theta)\rangle = M(\theta)\,|0\rangle^{\otimes n}\,, \tag{18}$$

where $M(\theta)$ represents the parameterized quantum circuit and n is the number of qubits. The circuit is defined as follows:

$$M(\theta) = \prod_{l=1}^{L}\left(\prod_{i=1}^{n} R_X(\theta_{l,i})\right)\left(\prod_{i=1}^{n-1}\text{CNOT}(i,i+1)\right), \tag{19}$$

where L is the number of layers, $R_X(\theta)$ is the X-rotation gate, and $\text{CNOT}(i,j)$ is the controlled-NOT gate with qubit i as control and qubit j as target. The expectation values of Pauli-Z observables are measured, which act as the final Q-Values:

$$q_t = (\langle Z_1\rangle, \ldots, \langle Z_n\rangle). \tag{20}$$

D. Agent

The QADQN agent is designed to make trading decisions based on historical financial data. The agent's architecture consists of several key components:

- Action space: $\mathcal{A} = \{sit, buy, sell\}$.
- Policy network (Q): A quantum-classical hybrid network for action selection.
- Target network ($\hat{Q}$): A periodically updated copy of the policy network for stable learning.
- Prioritized Replay memory: For experience replay and prioritized experience replay.

The QADQN agent employs an Upper Confidence Bound (UCB) policy for action selection [35], and integrates the Dual Thrust pre-populated actions from prioritized experience replay memory for the final action decision:

Action Selection Policy (UCB):

$$P(a|s) = \begin{cases} 1 - c + c\frac{\log(t)}{N_t(s)}, & \text{if } a = \arg\max_a Q(s,a) \\ \frac{c\log(t)}{N_t(s)}, & \text{otherwise} \end{cases}, \tag{21}$$

where c is a constant determining the exploration-exploitation trade-off, t is the total number of time steps, and $N_t(s)$ is the number of times state s has been visited up to time t.

Dual Thrust Strategy:

$$\begin{aligned} \text{Range} &= \max[\text{HH} - \text{LC}, \text{HC} - \text{LL}], \\ \text{BuyLine} &= \text{Open} + K1 \times \text{Range}, \\ \text{SellLine} &= \text{Open} - K2 \times \text{Range}, \end{aligned} \tag{22}$$

where Open represents the day's opening price, $K1$ and $K2$ are constants controlling market resistance levels against breaking BuyLine and SellLine, respectively. HH, LC, HC, and LL denote the highest high and lowest close prices over the previous periods [36].

E. Training Algorithm

The QADQN agent is trained using a combination of experience replay and prioritized experience replay. The training process is detailed in Algorithm 1.

Algorithm 1 QADQN Training Algorithm

```
Initialize replay memory D to capacity N
Initialize prioritized replay memory PD to capacity M
Initialize action-value function Q with random weights θ
Initialize target action-value function Q̂ with weights θ⁻ = θ
for episode = 1 to E do
    Initialize state S_1
    for t = 1 to T do
        Compute UCB action values UCB(S_t, a) for all actions a
        Select action a_t = arg max_a UCB(S_t, a)
        Execute action a_t and observe reward r_t and next state S_{t+1}
        Store transition (S_t, a_t, r_t, S_{t+1}) in D
        Sample transitions from D according to priorities
        Compute priorities for sampled transitions
        Perform gradient descent step w.r.t. θ on (y_j − Q(S_j, a_j; θ))^2
        Update priorities in PD based on TD error
        if t mod target_update_frequency == 0 then
            θ⁻ ← θ
        end if
    end for
end for
```

Lion (EvoLved Sign Momentum) optimizer [37] is used for the classical optimization step. It only tracks momentum and uses the sign operation to calculate updates, leading to lower memory overhead and uniform update magnitudes across all dimensions. Lion has shown improvements in training various models for different tasks, especially in networks with Attention layers, compared to optimizers like Adam and Adafactor. It is

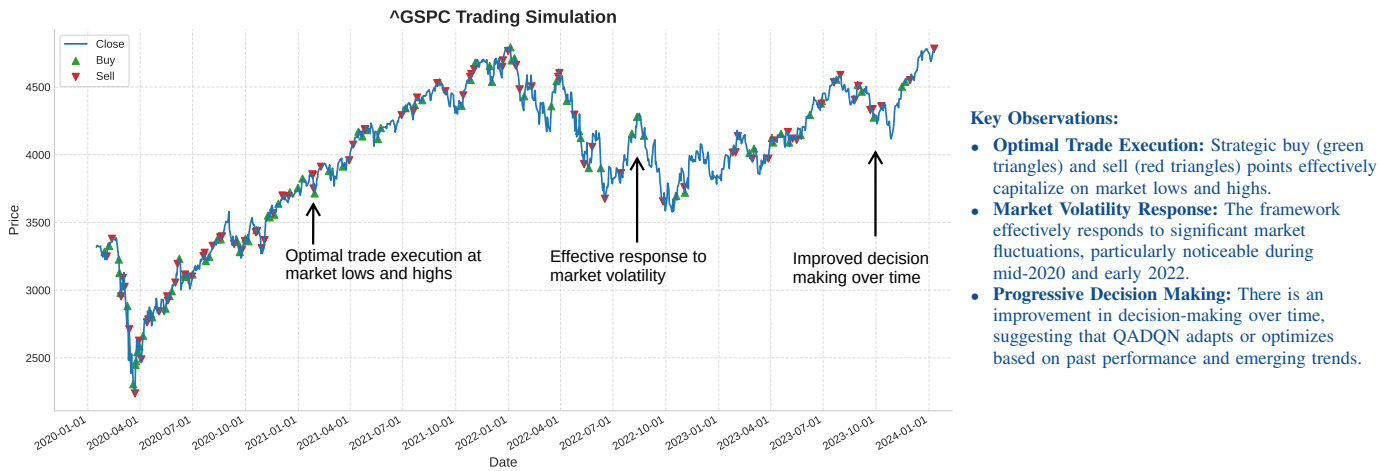

Fig. 3. QADQN Agent trading simulation for S&P 500 showcasing trading performance over time.

expressed as:

$$\theta_t = \theta_{t-1} - \alpha \cdot \text{sign}(m_t) - \alpha\lambda\theta_{t-1}, \tag{23}$$

where θ_t is the parameter at time step t, m_t is the exponential moving average of the gradient, α is the learning rate, λ is the weight decay coefficient and $\text{sign}(x)$ is the sign function. Due to the usage of sign operation as seen in equation 23, it balances the gradient history and current gradient weighting, leading to better global optimization. This makes the gradients smooth and continuous, leading to better decision-making by the QADQN. The loss function used is Huber loss as it proves advantageous in scenarios where observed rewards occasionally contain outliers—instances where unrealistically large negative or positive rewards may occur during training but not during testing, and it can be described as:

$$L = H(Q(S_t, a_t; \theta) - (r_t + \gamma \max_{a'} \hat{Q}(S_{t+1}, a'; \theta^-))), \tag{24}$$

where H is the Huber loss function. Unlike L2 loss, which could be significantly affected by such outliers, Huber loss maintains robustness by moderating the impact of these anomalies on the training process [38], [39]

IV. Results and Discussion

A. Experimental Setup

The experiments are conducted using the S&P 500 as the standard dataset with OHLC fetched from *Yahoo finance API* [40], with a training period from January 15, 2016, to January 15, 2020, and evaluation on two test periods: an overlapping period from January 16, 2019, to January 16, 2024, and a non-overlapping period from January 16, 2020, to January 16, 2024. The QADQN framework is implemented with 4 qubits for each quantum layer, a 24-day window size, a discount factor (γ) of 0.95, and 200 training episodes. The Dual Thrust strategy parameters are set to $k1 = 0.8$ and $k2 = 0.4$. The framework is trained using both standard experience replay and prioritized experience replay.

B. Performance Evaluation

The QADQN model's performance is evaluated using both visual analysis of trading actions and quantitative metrics derived from backtesting and is compared to the Buy & Hold Method and Deep Deterministic Policy Gradient (DDPG), which is an off-policy A2C model [41].

1) Trading Visualization

Fig. 3 presents a visual representation of the model's trading activities superimposed on the S&P 500 price chart for the non-overlapping test period. The green markers indicate buy actions, predominantly occurring at lower price points of the index, while the red markers denote sell actions, typically executed at higher price levels. This pattern suggests that the agent has developed a strategy that potentially capitalizes on price fluctuations, buying at relative lows and selling at relative highs. Such behavior indicates the model's ability to identify and exploit short-term price trends within the market.

2) Backtesting Results

Backtesting is performed using the Backtest library with a commission rate of 0.2%. The results for both test periods are summarized in Table I.

TABLE I
BACKTESTING RESULTS AND COMPARISON FOR QADQN FRAMEWORK FOR S&P 500.

	Metric	Overlapping Period	Non-Overlapping Period
QADQN	Return [%]	**78.91**	**71.26**
	Sharpe Ratio	**0.82**	**0.77**
	Sortino Ratio	**1.19**	**1.28**
	Max. Drawdown [%]	19.24	19.24
Buy & Hold	Return [%]	74.42	44.22
	Sharpe Ratio	0.67	0.52
	Sortino Ratio	1.07	0.85
DDPG	Return [%]	31.23	15.94
	Sharpe Ratio	0.42	0.36
	Sortino Ratio	0.81	0.79

C. Discussion

The QADQN framework demonstrates strong potential in applying quantum-enhanced RL to financial trading. The framework not only achieves substantial positive returns but also outperforms the Buy & Hold and DDPG benchmarks in

both test periods, particularly in the non-overlapping period.

Total Return: The QADQN framework achieves significant positive returns in both test periods, with 75.37% for the overlapping period and 66.82% for the non-overlapping period. Notably, the framework outperforms the buy-and-hold strategy in both cases, with a particularly substantial outperformance in the non-overlapping period.

Risk-Adjusted Performance: The Sharpe ratios of 0.78 and 0.75 for the overlapping and non-overlapping periods, respectively, indicate good risk-adjusted returns. The higher Sortino ratios (1.14 and 1.33) suggest that the framework manages downside risk very effectively, especially in the non-overlapping period, suggesting good generalization to unseen market conditions.

Maximum Drawdown: Both test periods exhibit a maximum drawdown of 19.24%, indicating the framework's ability to limit significant losses during market downturns.

V. Conclusion

This paper introduces a QADQN, which integrates a VQC within a traditional deep Q-learning network, aimed at emphasizing quantum computational advantages for financial trading decision-making, presenting a solution for investors to optimize trading strategies. Through rigorous literature review, framework development, and comprehensive empirical evaluation, our work marks a substantial advancement in quantum finance. The empirical results confirm the QADQN framework's robust performance, significantly surpassing standard frameworks and the buy-and-hold strategy in terms of total returns and risk-adjusted metrics during both test periods. This performance highlights the framework's effectiveness in adapting to diverse market scenarios and its capability to manage downside risks, evidenced by favorable Sharpe and Sortino ratios. Despite these promising outcomes, our study recognizes the need for broader testing across varied market conditions and addresses the challenges related to quantum circuit scalability in practical applications. Future research should thus focus on enhancing the scalability of the QADQN framework, incorporating more complex quantum circuits, and expanding its application to different asset classes. Additionally, exploring quantum noise impacts and the potential for quantum error correction within the framework could further solidify the practical deployment of quantum-enhanced RL in financial markets.

Acknowledgment

This work was supported in part by the NYUAD Center for Quantum and Topological Systems (CQTS), funded by Tamkeen under the NYUAD Research Institute grant CG008.

References

[1] F. Rundo *et al.*, "Machine learning for quantitative finance applications: A survey," *Applied Sciences*, vol. 9, no. 24, 2019.

[2] R. Y. Chou, H. Chou, and N. Liu, *Range Volatility: A Review of Models and Empirical Studies*. Springer New York, 2015, pp. 2029–2050.

[3] Z. Niu, C. Wang, and H. Zhang, "Forecasting stock market volatility with various geopolitical risks categories: New evidence from machine learning models," *International Review of Financial Analysis*, vol. 89, p. 102738, 2023.

[4] F. X. Diebold, R. Garcia, and K. Jacobs, "Predictive Modeling, Volatility, and Risk Management in Financial Markets: In Memory of Peter F. Christoffersen (Part I)," *Journal of Financial Econometrics*, vol. 18, no. 3, pp. 471–472, 09 2020.

[5] H. Yang *et al.*, "Deep reinforcement learning for automated stock trading: an ensemble strategy," in *Proceedings of the First ACM International Conference on AI in Finance*, ser. ICAIF '20. Association for Computing Machinery, 2021.

[6] W. Zhang and S. Skiena, "Trading strategies to exploit blog and news sentiment," *Proceedings of the International AAAI Conference on Web and Social Media*, 2010.

[7] A. L. Awad, S. M. Elkaffas, and M. W. Fakhr, "Stock market prediction using deep reinforcement learning," *Applied System Innovation*, vol. 6, no. 6, 2023.

[8] J. Zou *et al.*, "Stock market prediction via deep learning techniques: A survey," 2023.

[9] V. Stojkoski *et al.*, "Generalised geometric brownian motion: Theory and applications to option pricing," *Entropy*, vol. 22, no. 12, 2020.

[10] G. Ladde and L. Wu, "Development of modified geometric brownian motion models by using stock price data and basic statistics," *Nonlinear Analysis: Theory, Methods & Applications*, vol. 71, no. 12, pp. e1203–e1208, 2009.

[11] D. L. Quayesam, A. Lotsi, and F. O. Mettle, "Modeling stock price dynamics on the ghana stock exchange: A geometric brownian motion approach," 2024.

[12] M. Zamani *et al.*, "Collective behavior of stock prices in the time of crisis as a response to the external stimulus," 2022.

[13] S. Belkhale, Y. Cui, and D. Sadigh, "Data quality in imitation learning," *Advances in Neural Information Processing Systems*, vol. 36, 2024.

[14] S. Chen, W. Luo, and C. Yu, "Reinforcement learning with expert trajectory for quantitative trading," *arXiv preprint arXiv:2105.03844*, 2021.

[15] J. Biamonte *et al.*, "Quantum machine learning," *Nature*, 2017.

[16] K. Zaman, A. Marchisio, M. A. Hanif, and M. Shafique, "A survey on quantum machine learning: Current trends, challenges, opportunities, and the road ahead," *arXiv preprint arXiv:2310.10315*, 2023.

[17] N. Innan *et al.*, "Financial fraud detection using quantum graph neural networks," *Quantum Machine Intelligence*, vol. 6, no. 1, p. 7, 2024.

[18] N. Innan, M. A.-Z. Khan, and M. Bennai, "Financial fraud detection: a comparative study of quantum machine learning models," *International Journal of Quantum Information*, vol. 22, no. 02, p. 2350044, 2024.

[19] D. Herman *et al.*, "A survey of quantum computing for finance," *arXiv preprint arXiv:2201.02773*, 2022.

[20] E. Farhi, J. Goldstone, and S. Gutmann, "A quantum approximate optimization algorithm," *arXiv preprint arXiv:1411.4028*, 2014.

[21] D. J. Egger *et al.*, "Credit risk analysis using quantum computers," *IEEE Transactions on Computers*, vol. 70, no. 12, pp. 2136–2145, 2020.

[22] Rivera-Ruiz *et al.*, "Time series forecasting with quantum machine learning architectures," in *Advances in Computational Intelligence*, 2022.

[23] D. Venturelli and A. Kondratyev, "Reverse quantum annealing approach to portfolio optimization problems," *Quantum Machine Intelligence*, 2019.

[24] D. Dong *et al.*, "Quantum reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics, Part B (Cybernetics)*, vol. 38, no. 5, pp. 1207–1220, 2008.

[25] M. Franz *et al.*, "Uncovering instabilities in variational-quantum deep q-networks," *Journal of The Franklin Institute*, vol. 360, no. 17, pp. 13 822–13 844, 2023.

[26] H. Zhang, Q. Zhao, and C. Chen, "A light-weight quantum self-attention model for classical data classification," *Applied Intelligence*, 2024.

[27] E. B. Unlu *et al.*, "Hybrid quantum vision transformers for event classification in high energy physics," *Axioms*, vol. 13, no. 3, 2024.

[28] R.-x. Zhao, J. Shi, S. Zhang, and X. Li, "Qsan: A near-term achievable quantum self-attention network," *arXiv preprint arXiv:2207.07563*, 2022.

[29] A. Vaswani *et al.*, "Attention is all you need," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, 2017.

[30] S. Shi *et al.*, "A natural nisq model of quantum self-attention mechanism," *arXiv preprint arXiv:2305.15680*, 2023.

[31] S. Y.-C. Chen, C.-H. H. Yang, J. Qi, P.-Y. Chen, X. Ma, and H.-S. Goan, "Variational quantum circuits for deep reinforcement learning," *IEEE access*, 2020.

[32] O. Lockwood and M. Si, "Reinforcement learning with quantum variational circuit," in *Proceedings of the AAAI conference on artificial intelligence and interactive digital entertainment*, vol. 16, no. 1, 2020, pp. 245–251.

[33] A. Skolik, S. Jerbi, and V. Dunjko, "Quantum agents in the gym: a variational quantum algorithm for deep q-learning," *Quantum*, vol. 6, p. 720, 2022.

[34] J. Zou *et al.*, "A novel deep reinforcement learning based automated stock trading system using cascaded lstm networks," *Expert Systems with Applications*, 2024.

[35] W. Zhang *et al.*, "Reinforcement learning for stock prediction and high-frequency trading with t+1 rules," *IEEE Access*, vol. PP, pp. 1–1, 01 2022.

[36] X. Cheng *et al.*, "Mot: A mixture of actors reinforcement learning method by optimal transport for algorithmic trading," in *Pacific-Asia Conference on Knowledge Discovery and Data Mining*. Springer, 2024, pp. 30–42.

[37] X. Chen *et al.*, "Symbolic discovery of optimization algorithms," *Advances in neural information processing systems*, vol. 36, 2024.

[38] T. Wang and D. E. Chang, "Improved reinforcement learning through imitation learning pretraining towards image-based autonomous driving," in *2019 19th international conference on control, automation and systems (ICCAS)*, 2019.

[39] S. Mishra and A. Arora, "A huber reward function-driven deep reinforcement learning solution for cart-pole balancing problem," *Neural Comput. Appl.*, 2022.

[40] Y. Finance, "Yahoo finance api," 2024, accessed: 2024-07-18. [Online]. Available: https://finance.yahoo.com

[41] P. Mironowicz *et al.*, "Applications of quantum machine learning for quantitative finance," *arXiv preprint arXiv:2405.10119*, 2024.